

Memory Overload: Data Layout & Parallel Processing in Large-Scale Traffic Analysis

NYC DOT Real-Time Traffic Speed Feed · 46.2M records · C++17 + OpenMP · 3-Phase AoS → SoA Benchmark · CMPE 275 Mini 1A

System 1

46.2M

Records
Processed

79.5 s

Serial Ingest
(Phase 1)

18.2 s

Parallel Ingest
(8T)

4.4×

Ingest
Speedup

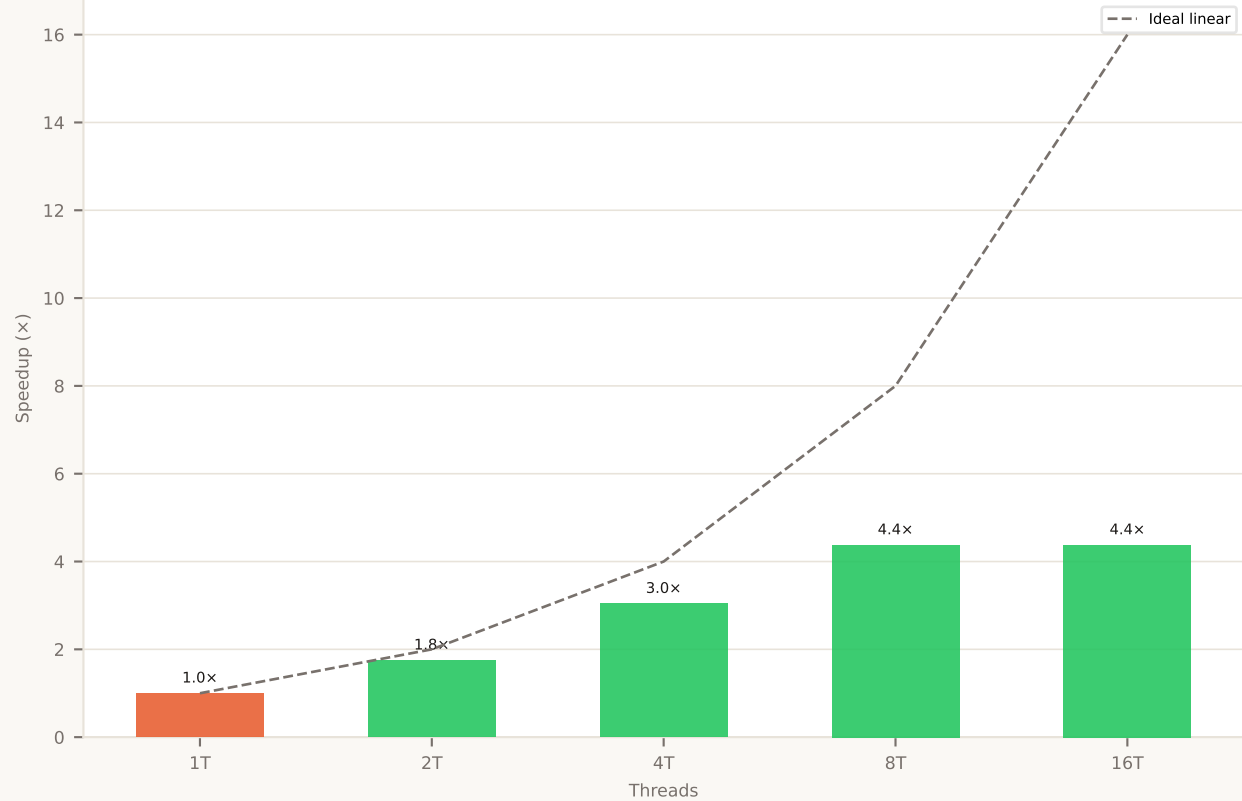
13.5×

Best Query
Speedup (W1)

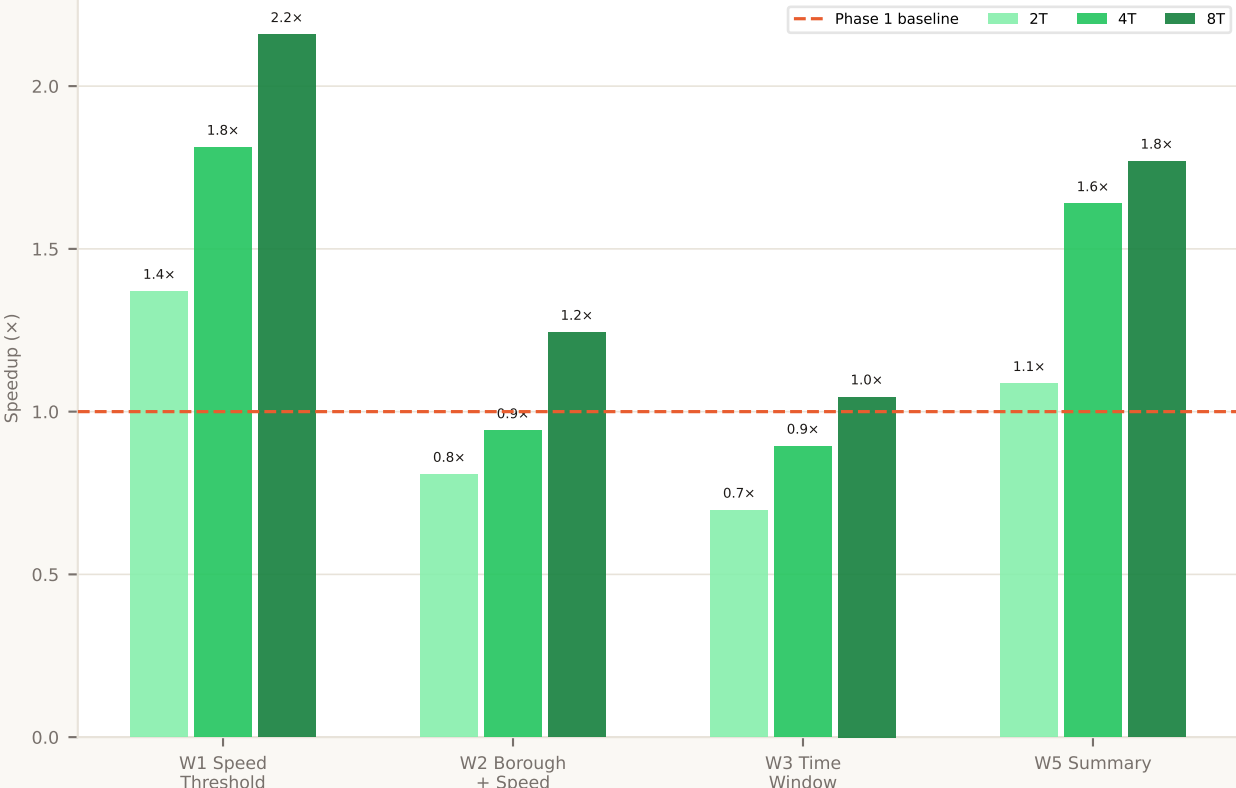
~46%

Memory
Reduction (SoA)

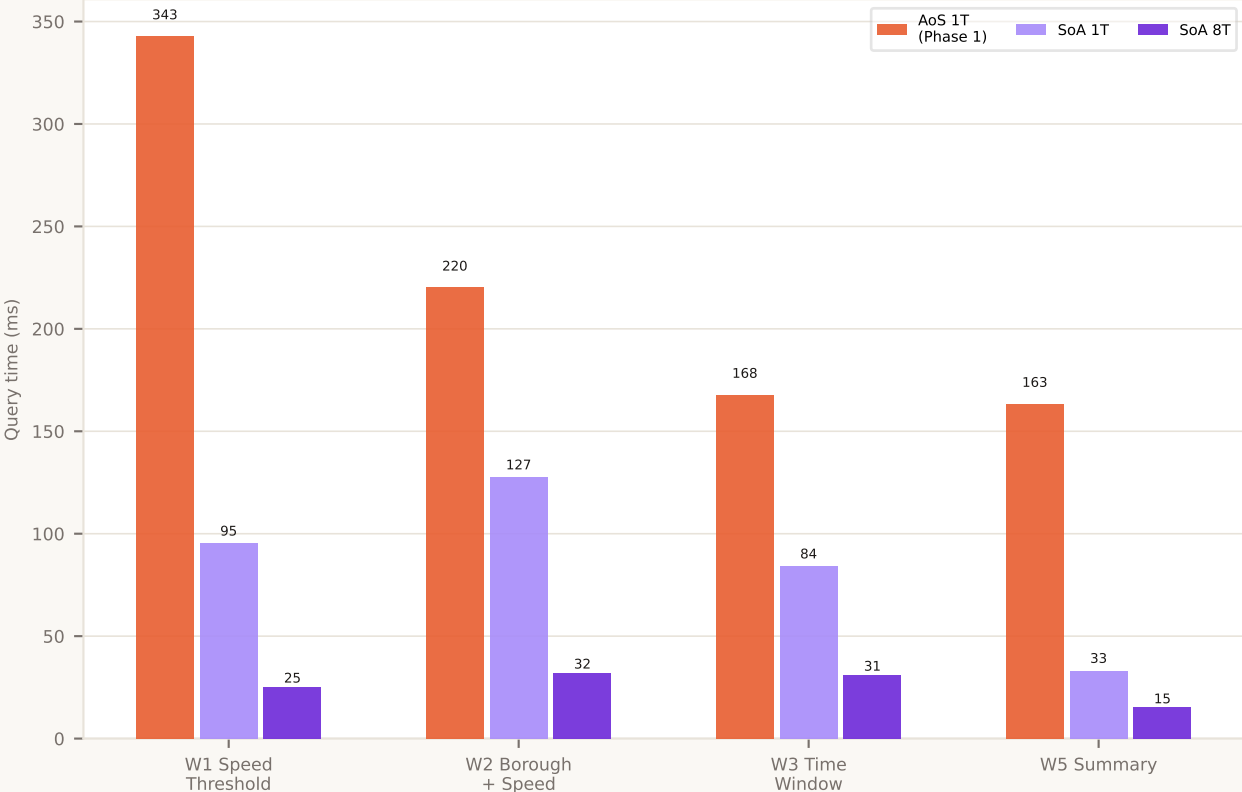
Ingest Speedup vs Serial (46M rows)



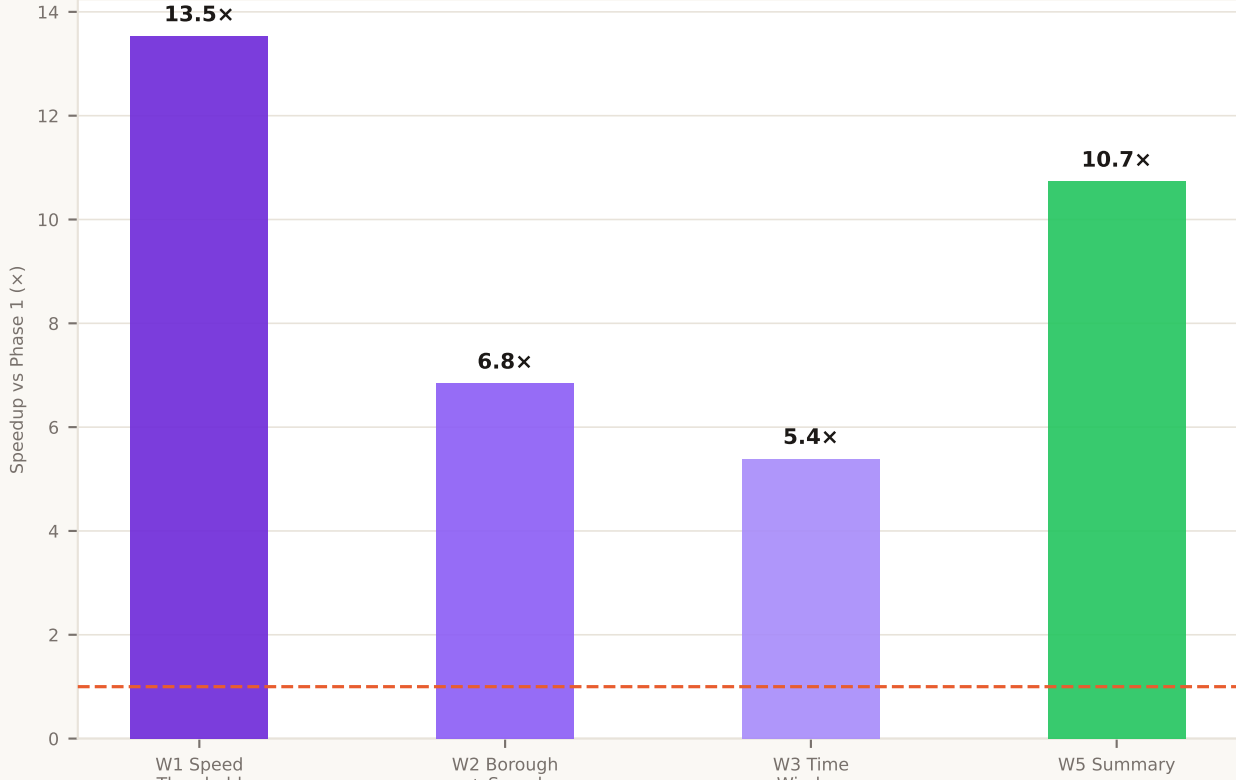
Phase 2 — Query Speedup vs Phase 1 Serial (AoS)



Phase 3 — Query Latency: AoS vs SoA (46M rows)



Total Speedup: Phase 3 (SoA + 8T) vs Phase 1 Serial



KEY FINDINGS

① Ingest Parallelism Critical at Scale

Serial CSV parsing dominates total runtime at 46M rows: 79.5 s vs 160–376 ms per query. Parallel mmap loader (8 threads) cuts this to 18.2 s — a 4.4× speedup — by memory-mapping the file and parsing N chunks in parallel. Speedup plateaus at 8T due to I/O bandwidth.

② SoA Layout Dominates Query Performance

AoS (Array-of-Structs) loads 104 bytes per record for every scan — including 64 bytes of std::string metadata. SoA stores one contiguous array per field, delivering 100% DRAM utilization. Layout alone yields 1.7× – 4.9× speedup before adding threads.

③ Combined: Up to 13.5× Total Speedup

SoA layout + 8 OpenMP threads achieves 5.4× – 13.5× over Phase 1 serial AoS. W1 (speed threshold) benefits most: its hot loop reads only 8 bytes/record. W4 (top-N, hash-map) gains least — the merge step serializes under contention.

④ Phase Comparison Summary

Phase 1	Serial AoS	1.0×	baseline
Phase 2	Parallel AoS	1.1–3.3×	query
	Parallel ingest	4.4×	
Phase 3	SoA serial	1.5–4.9×	layout
	SoA + 8T	5.4–13.5×	total

Dataset

NYC DOT Real-Time Traffic Speed Feed
46,264,669 records · 19 GB raw CSV
16-core Intel i9-12900 · 94 GB RAM
GCC -O2 -fopenmp · Linux 6.8